AutoBench Service — Admin Guide

Last modified: 2026-10-15T01:52:20Z

> Hand-maintained, unlike the generated `results/12run-*.md` files which stamp themselves. Bump the line above when you edit this guide.

How the AutoBench Service gets onto a cluster and stays trustworthy there: prerequisites, the environment that differs between platforms, the Helm chart, and the verification that actually proves the install. *Driving* the Service once it answers is a different document — `DEVELOPER_GUIDE.md`.

- **Every host below is a placeholder on `example.com`** — this repo is public, so it names platforms rather than endpoints. Route *shapes* are real (`<service>-<namespace>.apps.<cluster>.example.com`), so substitute your own domain and the recipes work unchanged.
- **The cluster names are samples, and so is every endpoint they stand for.** `ykt3` is the OpenShift cluster this project happens to run on, reached from the Internet; the KinD cluster is a laptop on an organisation intranet. Nothing here requires any of them, or a particular LLM service (§3.5) — they are the worked examples, and the chart is deliberately generic.
- **No secret value appears in this guide, and none should appear in your terminal.** Every script here reads credentials from the environment or from a `chmod 600` file and reports them only as a truncated SHA-256. If you find yourself about to `echo` one, don't.
- Two install paths exist and both are supported: the **Helm chart** at `deploy/helm/autobench` (§5) and the **raw manifests** under `deploy/` (§8). They render the same objects, and that equivalence is enforced by a check, not asserted.

In a hurry? §2.4 is the one command that tells you whether the cluster is ready, §5 is the install, §6 is the only verification that means anything.

Contents

- 1. What you install, and what you don't
  - How the pieces are laid out
- 2. Prerequisites
  - 2.1 Rossoctl v0.8.0 or later
  - 2.2 Workstation tooling
  - 2.3 Cluster-side checklist
  - 2.4 The preflight script
- 3. Environment variables
  - 3.1 Install-time — your shell or an env file, never a values file
  - 3.2 The Service pod
  - 3.3 The workload pods — injected, not configured
  - 3.4 KinD and OpenShift differ — and the differences fail silently
  - 3.5 The LLM gateway: two named profiles
  - 3.6 The `benchmarker` password: how it gets in, and how it is checked
  - 3.7 MLflow: already there, or installed by us — and how you find out which

- 4. The instance-config Secret
- 5. Installing with Helm
  - What the scripts install: one cluster
  - 5.1 OpenShift
  - 5.2 KinD
  - 5.3 Adopting an install made from the raw manifests
  - 5.4 The IBAC judge — installed and uninstalled with the release
  - 5.5 Upgrade, rollback, uninstall
- 6. Verifying the install
- 7. Symptoms that lie
- 8. Appendix: the raw-manifest path, and two KinD-only objects

1. What you install, and what you don't

The Service is a single stateless pod. It holds no benchmark data and no cluster credential: it authenticates to Rossoctl per request with its own ROPC identity, and Rossoctl performs every cluster operation server-side. Nothing here shells out to `kubectl` or `oc`.

object	name	created by	notes
Deployment	`autobench-service`	chart / `deploy/deployment.yaml`	1 replica, read-only root filesystem, arbitrary-UID safe
Service	`autobench-service`	chart / `deploy/service.yaml`	ClusterIP `:8080`, port named `http`
Route	`autobench`	chart (`platform: openshift`)	edge TLS; host generated as `autobench-<namespace>.apps.<cluster>`
HTTPRoute	`autobench`	chart (`platform: kind`)	attaches to the shared istio gateway
Secret	`autobench-instances`	**out-of-band**: `autobench-install.sh`, or you by hand (§4)	ROPC credentials and S3 keys — never passes through Helm values
RoleBinding	`autobench-service-mlflow-trace-writer`	chart (`mlflowTraceWriter.enabled`)	one per MLflow workspace namespace; lets the Service **write** its own spans. On by default in `values-openshift.yaml` — see the warning below
Deployment + Service	`ibac-judge`	chart (`ibacJudge.enabled`, off by default)	needed only by the plugin legs #5–#8 (§5.4)

The judge is in that table, rather than in the platform list below, on purpose: **Rossoctl ships none.** `rossoctl-platform-config` arrives with every `ibac.*` field empty, and a cluster with no benchmark workloads never needs one — so the judge exists for benchmarking, AutoBench installs it, and `helm uninstall` takes it away again (§5.4).

Everything else the Service depends on belongs to the platform and must exist **before** the install is useful:

- a Keycloak user (`benchmarker` by convention) with the **`rossoctl-operator` realm role**;
- `openai-secret` (`apikey`) and `hf-secret` (`hf-token`) in **every namespace you deploy benchmarks into** — `team1` unless you have changed the specs, since all 24 entries in `reference/run12_specs.json` name it;
- an OTEL collector that writes to an MLflow the Service can read (§3.4).

How the pieces are laid out

AutoBench separates the **Service** from the **workloads** it benchmarks. The Service routes every request by the caller's JWT issuer (`iss`) to an **instance**: a per-cluster record naming that cluster's Keycloak, its Rossoctl backend, and where its agents and MCP servers can be reached. The workloads land wherever that instance's Rossoctl puts them, on the Service's own cluster or on another one. Slide 7 of `AutoBench.pptx` draws the layout.

piece	where it lives	set by
AutoBench Service	one Helm release `autobench` in `rossoctl-system`	the chart (§5)
instances	one file per issuer in the `autobench-instances` Secret, next to the Service	§4
benchmark workloads	**every agent and MCP server in one namespace**, `team1`, on the instance's cluster	every leg of `reference/run12_specs.json`; preflight checks that namespace; the uninstaller's `--teams` defaults to it
model-gateway and HuggingFace keys	`openai-secret` and `hf-secret` in that namespace	provisioned out-of-band (§2)
OTEL collector	on the **workload** cluster; forwards the agents' spans to the MLflow beside the Service	the platform (§3.4)
MLflow	on the **Service's** cluster; the Service writes its own spans there and reads the traces back. Workspace `team1` on OpenShift	the instance file (§4)
S3	an external cloud service, shared by every instance	the instance file (§4)
IBAC judge	one per **workload** cluster, named in its `rossoctl-platform-config`	the chart (§5.4)
Keycloak	the workload cluster's realm `rossoctl`; the Service acts there as `benchmarker` (realm role `rossoctl-operator`), and any realm user can call it	the instance file's `service_credential`

What the install scripts set up, and what they leave to you, is in §5, "What the scripts install".

One Service serves several people: every run is attributed to the caller's own token, and its S3 keys carry the caller's username. Two rules keep shared use clean — never deploy the **same benchmark into the same namespace at the same time** (the workload names are fixed per benchmark, so the second deploy collides with the first), and run **one measurement campaign at a time**, because concurrent runs contend on the shared LLM gateway: pass rates stay valid, latency and cost do not.

The namespace is a request field (`namespace` on every deploy and run, `--namespace` on the CLI), not a constant. Using a second one needs its own `openai-secret` and `hf-secret`, and on OpenShift its MLflow

workspace in `mlflowTraceWriter.namespaces`.

2. Prerequisites

2.1 Rossoctl v0.8.0 or later

This is the hard one, because falling short of it does not error. An older operator accepts the deploy request the Service sends and **silently drops fields it does not know** — `k8sResourceLimits` among them — so the benchmark comes up in a shape nobody configured and every subsequent measurement is quietly off.

Read the version off the **backend**. The operator subchart carries its own, lower, version line, and comparing that one rejects a perfectly current cluster:

```
oc -n rossoctl-system get deploy rossoctl-backend \
  -o jsonpath='{.metadata.labels.app\.kubernetes\.io/version}{"\n"}'    # 0.8.0-rc.2
```

A release-candidate suffix counts: `0.8.0-rc.2` satisfies "≥ 0.8.0". Both `reference/preflight.py` and `reference/ocp-service-bootstrap.sh` assert this before doing anything else.

2.2 Workstation tooling

tool	needed for	note
`kubectl` (or `oc`)	everything	`oc` only for `Route` conveniences; `kubectl` can do it all
`helm` ≥ 3.8	§5	3.8 is the floor for the OCI/`--kube-context` behaviour used here
`jq`, `curl`	the bootstrap scripts	secrets move through `jq` via the *environment*, never argv
`uv`	every Python step of `autobench-install.sh`, `autobench-uninstall.sh` and the bootstrap scripts they call	**required**. They run Python only as `uv run --project <repo> --frozen python` (`reference/pyrun.sh`): the repo's own environment, built from `uv.lock` on first use, never the `python3` on PATH. There is no virtualenv to set up, and a workstation interpreter's version or missing PyYAML cannot change what the install does
`python3`	running `preflight.py`, `helm-parity-check.py` or `gen_toc.py` by hand	optional — `uv run python reference/…` does the same. `preflight.py` is stdlib-only and borrows the uv environment for the one check that parses YAML
`kind` + a container engine	the KinD path only	the image is built locally and `kind load`ed

2.3 Cluster-side checklist

Each row is a failure that has actually been shipped, and the right-hand column is why you cannot find it by reading logs at the layer where it hurts.

prerequisite	where	if missing
Rossoctl ≥ v0.8.0	`rossoctl-system`	fields dropped from the deploy request, silently (§2.1)

prerequisite	where	if missing
`agentruntimes` + `agentcards` CRDs	cluster	`/deploy` fails at the operator
`benchmarker` user with a **password credential**, **ROPC enabled** on the client	realm	token request 400s; on this realm a missing `firstName`/`lastName` yields `Account is not fully set up`. Checked in two tiers, because ROPC alone cannot distinguish a wrong password from an absent one (§3.6)
`rossoctl-operator` realm role on that user	realm	`/deploy` surfaces the operator's 403 as a **502**
`openai-secret` / `apikey`	`team1`	empty or foreign key ⇒ a 401 **per completion, mid-run**; the leg finishes with zeroes rather than failing
`hf-secret` / `hf-token` (may be empty)	`team1`	MCP pod `CreateContainerConfigError`, the agent then crash-loops against it, and the *run* reports 424
OTEL collector, HTTP receiver on **:8335**	`rossoctl-system`	an agent pointed at 4318 hard-fails into `CrashLoopBackOff`
an MLflow the collector **writes to** and the Service can **read**	varies (§3.4)	the run **passes** with `model: "unknown"` and every token count 0

On the collector port: 8335 and only 8335. Every cluster's `otel-collector-config` declares receivers on 4317/4318, and reading that ConfigMap will tell you to use 4318. It is wrong — the Deployment moves the HTTP receiver with a command-line override, so 4318 never listens:

```
oc -n rossoctl-system get deploy otel-collector \
  -o jsonpath='{.spec.template.spec.containers[0].command}'
# ["/otelcol-contrib","--config=/etc/otelcol-config/base.yaml",
#  "--set","receivers::otlp::protocols::http::endpoint=0.0.0.0:8335"]
```

The same principle applies to MLflow: read the target and the experiment id from the collector's own exporter block rather than choosing them. `traces_endpoint` and the `x-mlflow-experiment-id` header are what the *writer* uses, and an instance config that disagrees points the Service at an experiment nothing writes to.

2.4 The preflight script

`reference/preflight.py` checks every row of §2.3 against the live cluster. It makes **no cluster writes and no deploys**, and prints every credential as an 8-char SHA-256 prefix unconditionally, with no attempt to decide which values are sensitive. It is read-only with exactly one exception, the final MLflow section: that asks the deployed Service for `GET /mlflow/health`, which writes one synthetic trace to MLflow (§3.7). `--skip-mlflow-probe` turns it off, at the price of the install no longer being gated on the failure that is hardest to notice.

```
python3 reference/preflight.py --platform kind      --context kind-rossoctl
python3 reference/preflight.py --platform openshift --context <ykt3-ctx>
python3 reference/preflight.py --json                                  # machine-readable
python3 reference/preflight.py --pre-install [--ibac-judge] \
                               --values deploy/helm/values-openshift.yaml   # before an install
```

Without `--pre-install` the script audits an install that already exists, so it FAILS on objects the chart creates — the trace-writer RoleBinding first among them — when the release is absent. `--pre-install` reads what the chart *will* create off `helm template` of `--chart` with `--values` and counts that as present.

`--ibac-judge` adds the judge's pieces. `autobench-install.sh` passes `--pre-install` (and `--ibac-judge` when the judge is on) before Helm runs, and neither after it. On KinD, `--kind-mlflow` (only with `--pre-install`) does the same for the MLflow read path the installer is about to create (§5.2): an absent `mlflow-reader` and a collector still pointed at the OIDC-gated `mlflow` are what that install fixes, so they are not failures, and the instance config is compared against where the collector *will* export.

The target can come from the env file, as it does for the installer: `KUBE_CONTEXT`, `AB_PLATFORM`, `HELM_VALUES`, `NAMESPACE` and `TEAMS` stand in for `--context`, `--platform`, `--values`, `--namespace` and `--teams`, and a flag beats them. The header names the variable each one came from — `context=… ($KUBE_CONTEXT)` — and only when neither is given is the *current* kubectl context audited, marked `(current)`.

```
uv run python reference/preflight.py --env-file ~/.rossoctl-ykt3/autobench.env --pre-install
```

On OpenShift the MLflow is never installed by us, so `--pre-install` does not excuse its absence: the MLflow section asks whether it **answers**, before Helm runs (§3.7).

Exit status is 0 when nothing FAILed; warnings do not fail the run. Thirteen sections, in the order a request travels: tooling, cluster reachability, Rossoctl version and CRDs, namespaces, workload secrets, the collector, MLflow, ingress, the instance config, identity, the MLflow round trip, an audit of any existing install, and the local chart.

```
Workload secrets (per team namespace)
  ok    team1/openai-secret apikey — sha8 0239f193
  ok    team1/hf-secret hf-token — empty value (fine)

OTEL collector (the write half of the telemetry chain)
  ok    collector HTTP receiver on :8335 — read from the Deployment's command
  ok    collector traces_endpoint — http://mlflow-reader.rossoctl-
   system.svc.cluster.local:5000/v1/traces
  ok    collector x-mlflow-experiment-id — 0

MLflow (the read half — this is what turns a run into a token report)
  ok    MLflow traces API answers 200 — experiment 0, 1 trace(s) visible
  ok    collector exports to mlflow-reader
...
MLflow round trip (the only check that reaches the write half end to end)
  ok    MLflow tracking URL (as the Service resolves it) — http://mlflow-reader.rossoctl-
   system.svc.cluster.local:5000
  ok    MLflow credential mode — bearer_token
  ok    MLflow auth — bearer obtained via bearer_token
  ok    MLflow read — traces API answered; experiment 0 is non-empty
  ok    MLflow write — probe spans exported
  ok    MLflow round trip — probe trace readable after 2 attempt(s)
...
47 ok, 0 warning(s), 0 failure(s) — ready to install
A clean preflight is necessary, not sufficient: only a 1-task leg with a non-zero token row
   proves the whole chain.
```

Two things about how it reads the cluster are worth knowing, because each was a false alarm first:

- **Ports and targets come from live objects**, never from a manifest or a ConfigMap — the collector's port from the Deployment's `command`, MLflow's identity from the exporter.

- **`Route` has no CRD.** It is served by the aggregated openshift-apiserver, so `get crd routes.route.openshift.io` reports it missing on a cluster that plainly serves Routes. The script asks `api-resources --api-group=route.openshift.io` instead.

The **identity** section is the one that needs a credential from you, and it is not optional in practice: every `/deploy` the Service makes is a ROPC login as `benchmarker`, so a password that is absent, stale or blocked by a required action does not degrade the install — it makes every run fail with a 502 wrapping a 403. Pass it in and preflight logs in for real:

```
python3 reference/preflight.py --platform kind --context kind-rossoctl \
        --password-file ~/.rossoctl-kind/benchmarker.pass
```

With no password at all the script falls back to the one inside the instance Secret, which is the copy that actually matters. §3.6 covers the four intake routes, the two tiers of the check, and what each failure cause means.

3. Environment variables

There are three populations, and conflating them is how a key ends up in the wrong cluster. They are, in order: what you export **to run the installer**, what the **Service pod** reads, and what the operator injects into the **workload pods**.

3.1 Install-time — your shell or an env file, never a values file

Nothing in this table is a Helm value, a ConfigMap, or a committed file. Each is read by a script, used once, and hashed if it is reported at all.

Every one of them can come from an env file. `reference/autobench.env.template` lists them all. Copy it outside the repo and fill it in:

```
umask 077; mkdir -p ~/.rossoctl-ykt3
cp reference/autobench.env.template ~/.rossoctl-ykt3/autobench.env   # chmod 600 — looser is
   refused
reference/autobench-install.sh --env-file ~/.rossoctl-ykt3/autobench.env
```

`--env-file` is accepted by `autobench-install.sh`, `autobench-uninstall.sh`, both bootstrap scripts, `kind-post-setup.sh` and `preflight.py`.

It is repeatable, and **the last value wins**, lowest to highest:

1. the shell environment;
2. each `--env-file`, in command-line order (within one file, a later line beats an earlier one);
3. an explicit flag.

So a small per-run override file can be layered over a base file: `--env-file base.env --env-file override.env`.

The grammar is smaller than a shell's on purpose:

- `KEY=value`, with optional `export` and one matching pair of quotes stripped.

- Nothing is evaluated. `$VAR` and `$(…)` are stored exactly as written, so a password containing `$` survives.
- A bad line is reported by file and line number, never by its value.

The template's value lines are commented out, because an uncommented `KEY=` *is* an assignment and would blank a value your shell exported. `*.env` is gitignored.

variable	used by	notes
`AB_PLATFORM`, `KUBE_CONTEXT`	`autobench-install.sh`, `autobench-uninstall.sh`	**required**: `openshift` | `kind`, and the Service cluster's kubectl context. Never defaulted: the current context is not necessarily the cluster you mean. `--platform` / `--context` override
`CLUSTER`, `HELM_VALUES`, `IMAGE_TAG`	`autobench-install.sh`	`CLUSTER` is required on OpenShift (it derives the apps and Keycloak hosts); `HELM_VALUES` defaults to `deploy/helm/values-<platform>.yaml`; `IMAGE_TAG` defaults to the chart's `appVersion`
`KC_SERVICE_USERNAME`	both bootstrap scripts, `preflight.py`	the ROPC login the Service uses against Rossoctl (default `benchmarker`); `--username` overrides
`KC_SERVICE_PASSWORD` / `KC_SERVICE_PASSWORD_FILE`	both bootstrap scripts, `preflight.py`, `autobench-uninstall.sh`	**required, exactly one of the two.** The `_FILE` form names a `600` file holding the password, which keeps it out of the env file; both set is an error. Written into the instance file, never echoed. `--password-file` / `--password-stdin` / `--password` take precedence (§3.6)
`S3_ENABLED`	both bootstrap scripts, `autobench-install.sh`, `preflight.py`	**required, no default**: `true` publishes run artifacts, `false` publishes none and writes no `s3` block. Unset is a precheck failure — see §4
`S3_BUCKET`, `S3_REGION`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`	same	**required when `S3_ENABLED=true`**, shape-checked, then the key is proven by a signed read-only request (`reference/s3check.py`). Only the key id's `sha8` is ever shown
`S3_PREFIX`, `S3_ENDPOINT_URL`	same	optional: the prefix defaults to `<cluster>/` (OpenShift) or `kind/`; the endpoint is for S3-compatible stores only, a bare `https://host[:port]`
`IBAC_JUDGE`, `IBAC_JUDGE_KEY_FILE`, `IBAC_JUDGE_UPSTREAM_BASE`, `IBAC_JUDGE_MODEL`	`autobench-install.sh`	the judge for the plugin legs (§5.4); `--ibac-judge` sets the first. The base is a **base** URL and the model takes **no** `openai/` prefix — both are shape-checked
`KC_SERVICE_CLIENT_SECRET`	both bootstrap scripts	only if the Keycloak client is confidential
`KC_USER_PASSWORD`	`kind-post-setup.sh`	the password to *seed*; the same three flags override it, and it falls back to `~/.rossoctl-kind/benchmarker.pass` (`KC_CRED_FILE`)
`KC_ADMIN_PASSWORD`	`kind-post-setup.sh`, `preflight.py`	optional — read from the in-cluster `keycloak-initial-admin` Secret when unset. In `preflight.py` it enables the Admin-API tier of §3.6
`KC_ADMIN_USER`	`preflight.py`	optional, default `admin` — the master-realm admin the Admin-API tier logs in as

variable	used by	notes
KC_ISS	preflight.py	optional — the issuer to log in against, when neither --iss nor the instance file supplies one
LLM_PROFILE	both bootstrap scripts, kind-post-setup.sh, preflight.py, autobench-install.sh	intranet | internet — selects one of the two gateway variable sets (§3.5). Beats the values file's llmProfile, and autobench-install.sh passes it to the release, so a KinD cluster can run on the internet gateway without editing values-kind.yaml
INTRANET_LLM_* / INTERNET_LLM_*	reference/llm-profiles.sh, read by all of the above	the profiles themselves: base, model, key file, optional bypass list (§3.5)
BM_WORKLOAD_LLM_KEY	kind-post-setup.sh	the workload LLM key; falls back to the selected profile's key file, else ~/.rossoctl-kind/litellm.key (LLM_KEY_FILE)
WORKLOAD_LLM_API_BASE, WORKLOAD_LLM_MODEL	both bootstrap scripts	set by LLM_PROFILE when you use one; an explicit export wins. The gateway has **no default** on purpose: this repo is public
MLFLOW_URL, MLFLOW_EXPERIMENT_ID, MLFLOW_WORKSPACE, MLFLOW_TOKEN_SECRET	ocp-service-bootstrap.sh	the id and workspace default to what the collector exports
WORKLOAD_OTEL_ENDPOINT, WORKLOAD_OTEL_INSECURE, WORKLOAD_AGENT_RUNNER	both bootstrap scripts	see §3.4 on the runner
IMAGE, CLUSTER, KUBE_CONTEXT, REALM, CLIENT, KC_HOST	kind-post-setup.sh	plain overrides, no secrets

Two conventions here are deliberate and both cost real time when broken:

The LLM key is `BM_WORKLOAD_LLM_KEY`, not `OPENAI_API_KEY`. That name is commonly exported in a developer's shell profile for an unrelated provider, and the script writes whatever it finds into cluster Secrets. A namespaced name cannot be inherited by accident. (Inside the pod the value still arrives as `OPENAI_API_KEY` — the registry maps it from the Secret's `apikey`.)

Credential files are refused unless they are `600` or `400`, and they are meant to outlive the cluster: neither the `benchmarker` password nor the LLM key changes across a rebuild.

```
umask 077; mkdir -p ~/.rossoctl-kind
printf '%s' '<benchmarker password>' > ~/.rossoctl-kind/benchmarker.pass
printf '%s' '<llm key>'              > ~/.rossoctl-kind/litellm.key
```

3.2 The Service pod

The container reads exactly two environment variables from the Deployment, and both are set by the chart:

variable	value	why
SERVICE_INSTANCES_DIR	/etc/service/instances	where the autobench-instances Secret is mounted read-only

variable	value	why
SERVICE_PORT	8080	read by the entrypoint (uvicorn), not by the settings model

Everything else is a `SERVICE_`-prefixed setting with a working default, overridable through `extraEnv` in the values file. These are the ones worth knowing:

setting	default	when you would change it
SERVICE_LOG_LEVEL	INFO	DEBUG while diagnosing an auth or deploy failure
SERVICE_HTTP_TIMEOUT_SECONDS	30	a slow Rossoctl API
SERVICE_JWKS_CACHE_SECONDS	300	rarely
SERVICE_EXPORT_SETTLE_MAX_SECONDS	20	a fast run can be exported *before* its child spans land, yielding `model: "unknown"` and zero tokens; the Service re-reads until the record set stops growing, and this is that budget
SERVICE_EXPORT_SETTLE_INTERVAL_SECONDS	2	with the above
SERVICE_SPAN_REPORT_MAX_ROWS	200000	only to cap a pathological trace — the export is built in memory

```yaml
# values fragment
extraEnv:
  - name: SERVICE_LOG_LEVEL
    value: DEBUG
```

The settings model uses Pydantic's `extra="ignore"`, so a **misspelled `SERVICE_*` name is accepted and discarded without a word**. If a setting seems not to take effect, suspect the spelling before the code.

3.3 The workload pods — injected, not configured

You do not set these. The Service derives them per deploy from the benchmark definition (`src/autobench/benchmarks/registry.py`) and the instance config, which is why an agent's environment cannot drift from the run that measured it. They are listed so you can recognise them in a pod spec:

variable	source	note
OPENAI_API_KEY	`secretKeyRef` → `openai-secret` / `apikey`, or the deploy's `llm_key_secret`	never a literal value in the pod spec
HF_TOKEN	`secretKeyRef` → `hf-secret` / `hf-token`	gsm8k only; presence is the requirement, not content
OPENAI_API_BASE, LLM_API_BASE	the deploy's `llm_api_base`, else instance `workload_llm.api_base`	any inherited values are **dropped** and re-injected
AUTOBENCH_LLM_GATEWAY	the deploy's `llm_gateway`, else `workload_llm.gateway_label`, else `default`	the label the artifacts publish; from v1.36
EXGENTIC_DEFAULT_RUNNER	instance `workload_agent_runner`	§3.4

variable	source	note
`REQUEST_TIMEOUT`	fixed `120`, from image v1.35	litellm's per-call ceiling; unset it is 6000 s, so a stalled gateway call was never retried
`EXGENTIC_OTEL_ENABLED`	`true` when `workload_otel.enabled`	
`OTEL_EXPORTER_OTLP_ENDPOINT` / `_PROTOCOL` / `_INSECURE`	instance `workload_otel`	the endpoint is the `:8335` one

A key Secret per extra gateway (for gateway × model comparisons)

A deploy can name another LLM gateway, the Secret holding its key, and a label (`llm_api_base` / `llm_key_secret` / `llm_gateway`; Developer Guide §6.8). Nothing gets re-installed: the only cluster-side step is **one Secret per extra gateway**, in every namespace you deploy into. The Service cannot create the Secret itself, because it has no Kubernetes access.

Create it from a chmod-600 file, so the key never reaches argv or your shell history:

```
kubectl -n team1 create secret generic cb-key --from-file=apikey=$HOME/.rossoctl-llm/cb.key
```

The data key is `apikey` unless the deploy says `cb-key:<key>`. If the Secret is missing, the agent never becomes Ready, and the run's `424` names the Secret. To label the instance's own gateway in the artifacts, set `workload_llm.gateway_label` in the instance file; otherwise it is published as `default`. Developer Guide §6.8 has a measured example: the whole 12-run on KinD through cb and through ETE-Ext, leg by leg.

3.4 KinD and OpenShift differ — and the differences fail silently

The Service's own objects are nearly identical across platforms. What differs is everything it points *at*.

	KinD	OpenShift	if you get it wrong
LLM gateway	**not a platform difference** — either platform may use a service on the intranet or on the Internet, subject to the asymmetric rule in §3.5	ditto	no two services share a key table, so a key moved across does not fail closed — it 401s per completion, mid-run
`openai-secret`	a key issued by **that cluster's own** service	ditto	same
MLflow read path	`mlflow-reader` (§8), no auth	the cluster's own MLflow on `:8443` with a ServiceAccount bearer and `insecure_tls`	a refused read is invisible: the run passes, every token count is 0
MLflow experiment	`0`	1, workspace `team1` on ykt3	identical signature to the above
Keycloak dial	the **backchannel** service DNS — `iss` is unreachable in-cluster, since `*.localtest.me` resolves to pod loopback and there is no CoreDNS rewrite	the `iss` Route itself	JWKS and ROPC both fail at startup
collector endpoint	`http://otel-collector.rossoctl-system.svc.cluster.local:8335`	the same	unreachable collector ⇒ agent `CrashLoopBackOff`, surfacing as a 424 on the run

	KinD	OpenShift	if you get it wrong
pod security context	UID/GID/fsGroup pinned to 10001/0/10001	`runAsNonRoot` + seccomp only	pinning 10001 can be **rejected** when it falls outside the project's allocated UID range
ingress	`HTTPRoute` on the shared gateway	`Route`, edge TLS	—

On OpenShift the Service's own span export needs two more things, and each fails silently. Reaching MLflow is TLS; being allowed to write to it is RBAC. The Service reads MLflow to build `report.ndjson`, but it also *emits* four spans of its own — `Agent.Session` and its three children — straight to the OTLP endpoint. If either half is missing, the export fails, MLflow never receives `Agent.Session`, and a trace without that root span is **dropped** when the report is assembled: the run succeeds, reports `pass_rate 1.0`, and publishes a **zero-byte** `report.ndjson` and `token_report.ndjson`. Nothing errors. The measurement is simply not there.

half	what it needs	rendered by
TLS — an in-cluster MLflow serves a cert signed by the OpenShift service CA, absent from the default trust store	`OTEL_EXPORTER_OTLP_TRACES_CERTIFICATE=/var/run/secrets/kubernetes.io/serviceaccount/service-ca.crt`	`extraEnv` in `values-openshift.yaml`; `deploy/openshift/deployment-patch.yaml` for the raw path
RBAC — writes are authorized against the MLflow operator's ClusterRole *in the MLflow workspace namespace*	a RoleBinding per workspace, granting `mlflow-reader` `mlflow-operator-mlflow-integration`	`mlflowTraceWriter.enabled: true`

It must be the OTLP-specific variable. `REQUESTS_CA_BUNDLE` looks like the obvious choice and is a trap: botocore reads it too, so it redirects the S3 client's trust store at the same time and every artifact upload then fails to validate AWS's public cert — turning an empty report into no artifacts at all. `preflight.py` checks both halves, and matches the RoleBinding on the **grant** rather than its name, because a cluster set up before the chart existed has an equivalent binding under a different one. `mlflowTraceWriter.namespaces` lists MLflow **workspaces** on the Service's cluster, which are the same strings as the agents' namespaces when the workloads share that cluster. A workspace missing from the list reproduces the empty report for that workspace only.

`insecure_tls` is a read-side lever, and turning it off hardens nothing while breaking the read. The two halves above cover the **write**: with `insecure_tls: false` the OTLP exporter picks up `OTEL_EXPORTER_OTLP_TRACES_CERTIFICATE` and verifies the service-CA chain successfully. The **read** client has no equivalent. It is one of exactly two things — a throwaway `httpx.AsyncClient(verify= False)` when `insecure_tls: true`, or the shared client on the **default system trust store** when false — and nothing anywhere hands it the service CA. So against an in-cluster MLflow the read path has two states, unverified or broken, and `insecure_tls: true` is what keeps it working.

Measured on an OpenShift install, one probe, `experiment_id` pinned so it could not confound the result:

stage	`insecure_tls: true`	`insecure_tls: false`
`write` (OTLP, has the CA anchor)	ok	**ok** — the anchor genuinely verifies

stage	`insecure_tls: true`	`insecure_tls: false`
`read` / `round_trip` (httpx, has none)	ok	**FAIL** `CERTIFICATE_VERIFY_FAILED … self-signed certificate in certificate chain`

Two consequences. `insecure_tls: false` is not the hardened setting it looks like — it converts a working install into an empty-report install, which is §7's most over-subscribed symptom. And the verification it skips is on a **cluster-local `.svc` connection**, so what is actually exposed is the MLflow bearer token to an attacker who can already intercept in-cluster service traffic. That is a real gap and a small one; it stays open deliberately, and the trigger to close it is any MLflow read that starts crossing a cluster boundary or an edge Route, where the same `verify=False` would stop being an in-cluster concern.

`workload_agent_runner` deserves its own warning. Use `direct`, the default.

- **Why not `service`.** It hosts the agent behind its own server threads, and those read a trace context shared by the whole process and owned by whichever session started last.
- **What that does at `max_parallel_sessions` > 1.** A task's `chat` spans land on *another* task's row, or on none. The run passes and the pass rate is right, but the per-task tokens are wrong.
- **Measured.** A controlled A/B on kind on 2026-10-03: gsm8k, 10 tasks, p=4, one agent image, only the runner changed.
  - `direct` put all 10 spans on their own task.
  - `service` lost 3 tasks' spans and put 6 of the other 7 on the wrong task.
- **Two more failures `service` carries:** the agent's `cannot pickle '_asyncio.Task'` race, and a 30 s cap on each agent step (§7).
- **The value can still change.** The agent image is pinned to `:latest`, so an upstream rebuild could change which runner attributes correctly.

Detect a wrong value by **fingerprint, not by zeros**: a gsm8k task's input-token count is the same in every run, so a row carrying a count that belongs to a different task is the tell (§6).

3.5 The LLM gateway: two named profiles

The gateway is the one piece of configuration that is neither discoverable from the cluster nor shared between clusters, so it gets its own section — and it is configured as a **named profile** rather than as a set of per-cluster values, because the failure it causes is silent.

Any OpenAI-compatible **LLM or LiteLLM service** can serve a deployment; nothing in AutoBench requires a particular one, and the endpoints this project happens to use are examples, not part of the product. Your organisation's will be different services with different hosts, model catalogues and keys.

What decides which one a deployment may be pointed at is **the network the service sits on relative to the cluster**, not which Kubernetes the cluster runs. Both platforms occur in both places: an OpenShift cluster is often deployed on the organisation's intranet, and a KinD cluster runs on an Internet server as readily as on a laptop on the VPN. `platform` and `llmProfile` are therefore independent, and nothing derives one from the other.

The rule is **asymmetric**:

- An **Internet** cluster — KinD or OpenShift — must be configured with the access credentials for an LLM or LiteLLM service **on the Internet**. An intranet service is not routable from it.
- An **intranet** cluster — KinD or OpenShift — can be configured with the access credentials for an LLM or LiteLLM service **on the intranet or on the Internet**, whichever it is permitted to reach. So an intranet cluster has a genuine choice, and the profile records which way it went.

	`intranet`	`internet`
what it points at	an LLM or LiteLLM service on the organisation's **intranet**	one on the **Internet**
which clusters may declare it	intranet clusters only — KinD or OpenShift	any cluster, on either network
why not the other	an Internet cluster cannot route to an intranet service — connect times out	an intranet cluster *may* use this; it is only barred where egress to the Internet is
model catalogue	ids from *that* service	**not** the same ids
key table	its own	its own — this is the whole problem

The two variable sets

Each profile is one set of environment variables, distinguished only by prefix, defined in `reference/llm-profiles.sh`:

`intranet`	`internet`	what it is
`INTRANET_LLM_API_BASE`	`INTERNET_LLM_API_BASE`	the gateway origin — scheme + host, no path
`INTRANET_LLM_MODEL`	`INTERNET_LLM_MODEL`	a model id from **that** gateway's catalogue
`INTRANET_LLM_KEY_FILE`	`INTERNET_LLM_KEY_FILE`	`chmod 600` file holding that gateway's key (default `~/.rossoctl-llm/<profile>.key`)
`INTRANET_LLM_NO_PROXY`	`INTERNET_LLM_NO_PROXY`	optional explicit egress-proxy bypass list; otherwise built for you

`LLM_PROFILE=intranet|internet` (or `--llm-profile`) selects one, and the scripts copy it into the canonical `WORKLOAD_LLM_API_BASE` / `WORKLOAD_LLM_MODEL` / `LLM_KEY_FILE` they already read. An explicitly exported `WORKLOAD_LLM_*` still wins, so nothing that worked before behaves differently.

Bases and model ids are not secrets, so keep them in a file instead of re-exporting per shell. The **keys stay out of it**, in their own per-profile files:

```
mkdir -p ~/.rossoctl-llm
cat > ~/.rossoctl-llm/profiles.env <<'EOF'
INTRANET_LLM_API_BASE=https://<intranet LLM service host>
INTRANET_LLM_MODEL=openai/aws/claude-haiku-4-5
INTERNET_LLM_API_BASE=https://<Internet LLM service host>
INTERNET_LLM_MODEL=openai/Azure/gpt-5-mini-2025-08-07
EOF
umask 077
printf '%s' '<intranet service key>' > ~/.rossoctl-llm/intranet.key
printf '%s' '<Internet service key>' > ~/.rossoctl-llm/internet.key
```

Per-profile key files are the point: no two services share a key table, so one file holding "the" key is exactly how a key gets used against the service that never issued it.

Selecting a profile for a deployment

The profile is declared **once**, as `llmProfile` in the chart values file, and everything else reads it from there:

```
# deploy/helm/values-kind.yaml            # deploy/helm/values-openshift.yaml
platform: kind                            # platform: openshift
llmProfile: intranet                      # llmProfile: internet
```

```
reference/ocp-service-bootstrap.sh  --llm-profile internet --cluster ykt3 --context <ctx> ...
LLM_PROFILE=intranet reference/kind-post-setup.sh
python3 reference/preflight.py --values deploy/helm/values-kind.yaml --context kind-rossoctl
```

The chart **does not** render the gateway into the Service pod, and cannot: the Service reads the gateway from `workload_llm.api_base` in the instance-config Secret (§4) — not from its own environment — and injects `OPENAI_API_BASE` / `LLM_API_BASE` onto each workload pod at deploy time (§3.3). So `llmProfile` is a **declaration**, and it does two things that matter:

- the bootstrap scripts take the profile from it, so the instance file they write cannot disagree with the release installed beside it;
- `preflight.py` **FAILS** when the live instance file's `api_base` is not that profile's base:

```
FAIL  keycloak.localtest.me_8080.json: LLM gateway matches the internet profile — instance
   points
      at <internal host>, the internet profile declares <external host> — separate key tables,
   so
      this 401s per completion mid-run rather than failing at deploy
```

That is the entire point of naming the profiles: it converts a mid-run 401 storm, whose only visible symptom is a leg that finishes with zeroes, into a pre-install error. `helm` rejects a mistyped profile for the same reason — `llmProfile: intranett` fails the render rather than being quietly ignored downstream.

Four details that have cost time

- **The key never travels in the instance file.** It lives only in `openai-secret` / `apikey` in each workload namespace, and reaches the pod as `OPENAI_API_KEY` through a `secretKeyRef`.
- **The proxy-bypass list is part of the gateway's configuration, not a separate concern.** Where the cluster injects an egress proxy, an in-network gateway base must be excluded from it or every completion leaves through a proxy that cannot reach it. `workload_llm.no_proxy` (and `disable_proxy: true`, which injects an empty `HTTP_PROXY` / `http_proxy`) are rendered onto the **agent** pod only. Both bootstrap scripts now build the list — loopback, the gateway host, the collector, and Keycloak's in-cluster service.
- **`WORKLOAD_LLM_BASE` was never a second setting.** It is an accepted alias for `WORKLOAD_LLM_API_BASE` in the KinD script, kept only so existing invocations keep working; the canonical name matches the instance-file field (`workload_llm.api_base`) on both platforms.
- **Do not verify a gateway with `curl`.** LiteLLM strips the `openai/` prefix, so the configured model name 403s by hand, and reasoning models reject `max_tokens` with a 400 (`max_completion_tokens` is the accepted field). The only trustworthy probe is a 1-task `gsm8k` leg (§6).

3.6 The `benchmarker` password: how it gets in, and how it is checked

This credential is load-bearing **per run**, not per install. The Service holds no cluster credential of its own: each `POST /benchmarks/{name}/deploy` is a fresh ROPC password grant as `benchmarker` against the realm, and the resulting token is what Rossoctl authorises. An install can therefore be completely healthy — pod Running, `/healthz` 200, `GET /benchmarks` listing every benchmark — and still fail every single run with a 502 that wraps Keycloak's 403.

There is no Keycloak object scoped to `team1`, incidentally: `team1` is a Kubernetes namespace, and the authorisation artifact for deploying into it is the **`rossoctl-operator` realm role** on the user in the `rossoctl` realm. So "is the password set for team1" is really five questions — does the user exist, is it enabled, does it have a `password` credential, is its profile complete enough for the realm to issue a token, and does it carry that role — and the checks below answer them separately.

Four ways in, in precedence order

Every script that needs the password takes the same four, most explicit first:

route	used as	notes
`--password-file F`	`preflight.py`, both bootstrap scripts, `kind-post-setup.sh`	**preferred.** The value never reaches argv or an environment. The file must be `600` or `400` or it is refused
`--password-stdin`	same	for a pipeline or a secret manager: `pass show … | script --password-stdin`
`--password P`	same	accepted, and warns every time: argv is world-readable through `ps` and `/proc/<pid>/cmdline`, and it lands in your shell history
the environment	`KC_SERVICE_PASSWORD` or `KC_SERVICE_PASSWORD_FILE` (bootstrap, preflight), `KC_USER_PASSWORD` (`kind-post-setup.sh`)	the original route, plus a `_FILE` spelling that names a `600` file — the form an env file (§3.1) wants. Setting both is an error, not a guess

Two conveniences on top: `kind-post-setup.sh` falls back to `~/.rossoctl-kind/benchmarker.pass` (`KC_CRED_FILE`), so on KinD no credential flag is ever needed; and `preflight.py` falls back to the `service_credential` inside the live `autobench-instances` Secret, so it can check an install whose password you do not have to hand.

Only the **source** is printed, alongside an 8-char SHA-256 prefix of the value:

```
Identity — the benchmarker credential every /deploy authenticates with
  ok    benchmarker password from file ~/.rossoctl-kind/benchmarker.pass — user benchmarker,
    sha8 8bb6f116
```

That hash is also how the supplied password is compared against the Secret's copy without either being shown. **A mismatch is a FAILURE, not a warning** — the Secret's copy is the one the Service presents, so a green preflight next to a Service that 502s on every deploy would be worse than no check at all.

Because a trailing newline is welded onto the value by every editor, the file readers take the **first line only**. A password with `\n` on the end fails ROPC with exactly the message a wrong password gives.

Two tiers, because ROPC cannot tell three failures apart

Keycloak answers `invalid_grant` / "Invalid user credentials" identically for a **wrong password**, a user with **no password credential**, and **no such user**. Those need three different fixes, so `preflight.py` asks the Admin REST API first when it can, and the ROPC login second:

```
  ok    user benchmarker exists in realm rossoctl — id e7557a3c-0a9f-4427-9d1a-3ab69727abc4
  ok    password credential set on benchmarker — credential type(s): password
  ok    ROPC login as benchmarker — against http://keycloak.localtest.me:8080/realms/rossoctl
  ok    realm role rossoctl-operator
```

The admin tier needs `KC_ADMIN_PASSWORD`, or an in-cluster `keycloak-initial-admin` Secret (which is how KinD ships — nothing to supply there). Without one it is skipped with a note and the ROPC login stands alone as the behavioural check. When the tier *has* run, its finding is threaded into the failure text, which is the difference between a guess and an answer:

```
FAIL  ROPC login as benchmarker — 'Invalid user credentials' — a password credential IS set on
   this
      user (checked above), so the password given to this script is not the one Keycloak holds.
```

The causes and their fixes:

cause	what it means	fix
`Invalid user credentials`, password credential **is** set	the password you supplied is not the one Keycloak holds	supply the right one, or re-seed with `reference/keycloak-ensure-user.sh`
`Invalid user credentials`, **no** password credential	the user exists with no password at all	`reference/keycloak-ensure-user.sh`
`Invalid user credentials`, **no such user**	not in this realm	`reference/keycloak-ensure-user.sh`; check you have the right realm
`Account is not fully set up`	the password may be **correct** — a required action is pending, or `firstName`/`lastName` are unset, which this realm's user profile demands before it issues a token	`reference/keycloak-ensure-user.sh`, which fills `firstName`/`lastName`/`email` where unset and clears required actions
`unauthorized_client` / `invalid_client`	the client is confidential, or has `directAccessGrantsEnabled: false`	`KC_SERVICE_CLIENT_SECRET`, or `keycloak-ensure-user.sh`, which enables Direct Access Grants idempotently
login ok, role missing	every `/deploy` will 403	`reference/keycloak-ensure-user.sh --realm-role rossoctl-operator`

Every row above has the same fix command, and preflight prints it under the failure, filled in for the realm it checked. Neither secret appears in it; both stay placeholders. A run with no `benchmarker` password at all can't name the realm yet, so it points at the script's `--help` instead:

```
FAIL  ROPC login as benchmarker — 'Invalid user credentials' — and no such user exists in the
 realm (see above)
     To create or repair benchmarker (idempotent; fixes every failure above):
       KC_ADMIN_PASSWORD=<Keycloak admin password> KC_USER_PASSWORD_FILE=<chmod-600 file> \
         reference/keycloak-ensure-user.sh --server https://keycloak.example.com --realm
 rossoctl \
         --client rossoctl --username benchmarker --realm-role rossoctl-operator --verify
     then give the installer the same file: KC_SERVICE_PASSWORD_FILE=<that file>.
     On KinD the admin password is in Secret keycloak-initial-admin (namespace keycloak).
```

The script is idempotent. It creates the user if it is absent, enables it, fills `firstName` / `lastName` / `email` where unset (keeping any that are set), clears pending required actions, sets the password permanently, grants the role and turns on Direct Access Grants for the client. With `--verify` it then logs in the way the Service will. The installer does not run it itself: creating an identity in someone's Keycloak needs that Keycloak's admin password, and that should be handed over on purpose, not taken.

Where in the flow each script checks

- **`kind-post-setup.sh`** seeds the user through `keycloak-ensure-user.sh --realm-role rossoctl-operator --verify`, which also fills `firstName` / `lastName` / `email` where unset and clears pending required actions — so the login it verifies is the one the Service will make.
- **`kind-service-bootstrap.sh`** verifies at the end of that chain, immediately before it writes the password into the instance file, and **dies** rather than writing an unusable one. It also warns when the token carries no `rossoctl-operator` role. `SKIP_CRED_CHECK=1` writes the file anyway — only useful when Keycloak is not reachable from where you are running.
- **`ocp-service-bootstrap.sh`** reports the source as a precheck row and logs in as another, alongside the Rossoctl-version and gateway-profile checks.
- **`preflight.py`** is the read-only pre-`helm` gate, and the only one with the admin tier.

3.7 MLflow: already there, or installed by us — and how you find out which

MLflow is the one dependency whose *existence* varies by cluster rather than by configuration. On OpenShift it is normally **pre-installed** — `rossoctl-deps` or RHOAI puts it there, SAR-gated on `:8443`, owned by whoever installed it. On KinD it normally has to be **installed by us**, because the MLflow `rossoctl-deps` ships runs mlflow-oidc-auth and rejects both the collector's export and the Service's read (§8). Neither is a rule: a KinD cluster on an Internet server may well be pointed at an MLflow that already exists, and then installing a second one is wrong.

So the install path **checks** instead of assuming:

script	what it does now
`autobench-install.sh --install-mlflow auto|always|never` (KinD)	the same three modes through `reference/kind-mlflow.sh`, plus an **ownership record**, so `autobench-uninstall.sh` removes the reader only when this installed it (§5.2)
`autobench-install.sh --install-mlflow auto|always|never` (OpenShift)	creates the credential the default shape reads — `sa/mlflow-reader` and `secret/mlflow-reader-token` — through `reference/ocp-mlflow.sh`, with the same kind of record (§5.1). Nothing is created when another shape below is declared

script	what it does now
`kind-post-setup.sh --install-mlflow auto|always|never`	`auto` (the default) applies `deploy/kind/mlflow-reader.yaml` only when nothing is already serving `MLFLOW_URL`; it logs which way it went and why. `never` is for a cluster whose MLflow is external — the collector is still repointed at `MLFLOW_URL`, which is the half that actually matters
`ocp-service-bootstrap.sh`	resolving to **no MLflow credential at all is a precheck failure**, not a warning, so an install never proceeds with an empty `bearer_token`
`kind-service-bootstrap.sh`	reads the experiment id off the collector, and a lookup that **cannot run is fatal**. It never guesses `0`: it runs in the uv environment of §2.2, or takes `--experiment-id N`
`preflight.py` (OpenShift)	**asks whether the pre-installed MLflow answers at all**, even with `--pre-install`, so the install stops before Helm rather than after it — see below
`autobench-cli mlflow-health`	the gate. One authenticated `GET /mlflow/health` against the deployed Service
`preflight.py`	calls that endpoint for both platforms; `--skip-mlflow-probe` opts out

On OpenShift: is it there at all?

"Normally pre-installed" is not "installed". The RHOAI component can be disabled, its pod can be unscheduled, or the URL the instance config names can be stale, and every one of those still lets a run **pass** — with an empty token report. So preflight checks, for the URL the collector writes to and each URL the Service will read from (the instance configs' `tracking_url`, or `MLFLOW_URL` / the RHOAI default with `--pre-install`), one target per distinct Service:

```
MLflow (the read half — this is what turns a run into a token report)
  ok    MLflow answers (collector writes) — mlflow.redhat-ods-applications:8443 answered (...)
```

The probe is one `GET /` through the API server's service proxy (`kubectl get --raw /api/v1/namespaces/<ns>/services/https:<svc>:<port>/proxy/`). It makes no write, and it sends no MLflow credential on purpose: MLflow's own 401 or 404 is exactly the answer wanted, proof that a server is there and speaking HTTP. What the API server says *itself* is what fails the check:

API server reply	means	result
`services "mlflow" not found`	MLflow is not installed at this URL	FAIL
`no endpoints available for service`	the Service exists, but no pod behind it is Ready	FAIL
`no service port N found`	the URL's port is wrong	FAIL
`error trying to reach service`	the pod did not answer	FAIL
no reply within 30 s	nothing answered at all	FAIL
`forbidden … services/proxy`, or `Unauthorized`	this kubeconfig cannot ask — log in again, or ask for `get services/proxy`	WARN
anything else, any HTTP status	MLflow answered	ok

A URL outside the cluster (an external MLflow) is SKIPPED, since only the round trip can reach it. None of this proves the Service's credential or network path. That is still the round trip's job, which needs the Service running, so after the install `preflight.py` runs it as before.

The credential is supplied, not discovered

A pre-installed MLflow is authenticated by whoever installed it, so its credential cannot be read off the cluster the way the experiment id and workspace can (§2.3). `ocp-service-bootstrap.sh` therefore exposes all four shapes the Service itself resolves, in this precedence — pick **one**:

flag(s)	instance config	for
`--mlflow-bearer-file F`	`bearer_token`	a pre-obtained bearer, read in-process from a `600`/`400` file
`--mlflow-token-secret S` (the default, `mlflow-reader-token`)	`bearer_token`	a ServiceAccount-token Secret in `rossoctl-system`
`--mlflow-username U --mlflow-password-file F [--mlflow-oauth-url URL]`	`username` + `password`	the OpenShift OAuth challenge flow an RHOAI oauth-proxy expects
`--mlflow-client-id ID --mlflow-client-secret-file F --mlflow-token-url URL`	`client_id` + `client_secret`	an MLflow fronted by Keycloak
`--mlflow-no-auth`	the ignored `bearer_token` placeholder of §4	declaring the MLflow unauthenticated — what KinD's `mlflow-reader` is, and the only way to install without a real credential

Values come from files or the cluster, **never argv**, and are displayed only as an 8-char SHA-256 prefix, the same discipline as §3.6. Half a shape is an error rather than a fallback: `--mlflow-username` without `--mlflow-password-file` stops the script, because quietly reverting to the token Secret would install a credential nobody chose. `kind-service-bootstrap.sh` takes `--mlflow-no-auth` with the same spelling; without it, it still *infers* no-auth from an `mlflow-reader` tracking URL, and now says so in the log.

Why the check is a round trip

`GET /mlflow/health` is authenticated as `benchmarker`, runs **four independent stages**, and always answers 200 — the verdict is in the body, because "the probe could not run" and "MLflow is broken" are different findings:

stage	what it proves
`auth`	a bearer could be minted at all, through `auth/mlflow.py` — the Service's own resolution, not a re-implementation. `credential_mode` names which shape was selected: `bearer_token`, `openshift_oauth`, `client_credentials` or `none`. A declared no-auth MLflow reads as `bearer_token`, because that is what the placeholder selects; `none` means nothing was configured, and it fails
`read`	the traces API answers. This is the half a token report reads
`write`	the OTLP exporter posted one synthetic trace **without logging an error**
`round_trip`	that trace came back out of the read side again

The last two are the reason this exists. The two failures that shipped zero-byte reports on 2026-09-30 (§3.4) were both on the **write** half, and the read path answered 200 throughout — reads go through `httpx` honouring `insecure_tls`, while the span exporter is a different library needing a real trust anchor and an RBAC grant. A read-only check would have passed on both clusters. Worse, the OTLP exporter never raises: `BatchSpanProcessor` runs it on a worker thread and logs the failure, which is why the endpoint captures the

`opentelemetry` logger for the probe's duration and returns the cause to you instead of leaving it in a pod log. A missing RBAC grant reports `Failed to export span batch code: 403, reason: Forbidden`; a missing TLS anchor reports the exporter's terminal line with the TLS cause spliced in, naming `CERTIFICATE_VERIFY_FAILED`.

The two read differently because a 403 is not retried and a TLS failure is: the exporter logs a retryable cause at `WARNING`, once per attempt, and leaves its terminal `ERROR` generic. The endpoint therefore watches both levels — without that, a missing TLS anchor would come back saying only `Failed to export span batch due to timeout, max retries or shutdown.` and naming no cause at all.

```
BM_BASE=https://autobench-rossoctl-system.apps.example.com \
BM_PASSWORD_FILE=~/.rossoctl-ykt3/benchmarker.pass \
  uv run autobench-cli mlflow-health          # exit 8 = not healthy; --no-round-trip reads only
```

The probe **cannot pollute a report**, which is what makes it safe to run against a live install: every report is filtered to its own run's session ids, and the probe's is a fresh uuid. It also costs no LLM gateway call. It does write one trace per call, so `?round_trip=false` exists for a cheap read-only check.

4. The instance-config Secret

One JSON file per issuer, keyed by `iss`, mounted read-only at `/etc/service/instances`. **The chart never creates it**: it carries the ROPC service credential and the S3 keys, so it is generated out-of-band and must never pass through Helm values or a values file. `autobench-install.sh` (§5) generates it and writes the Secret for you. This section is what that script runs, and what to run by hand.

One script per platform, so the file has reproducible provenance instead of being hand-assembled:

```
# OpenShift — prechecks the whole chain, S3 included, before writing
reference/ocp-service-bootstrap.sh --cluster ykt3 --context <ctx> \
  --env-file ~/.rossoctl-ykt3/autobench.env \
  --apps-domain apps.ykt3.example.com \
  --llm-api-base https://<external-gateway>/v1 \
  --out-dir instances/
# KinD — verifies the credential before writing it into the file
reference/kind-service-bootstrap.sh --context kind-rossoctl \
  --env-file ~/.rossoctl-kind/autobench.env \
  --copy-from instances/keycloak.localtest.me_8080.json \
  --out-dir instances/
```

Both take the password from `--password-file`, `--password-stdin`, `--password`, or `KC_SERVICE_PASSWORD[_FILE]` in the environment or the env file. Both default the user to `benchmarker` (`--username` overrides); see §3.6.

S3 is declared, never discovered. Both scripts refuse to write a file until `S3_ENABLED` says which install this is:

`S3_ENABLED`	precheck	the file gets
unset	**FAIL** `s3 declared`, pointing at the template	nothing; the script stops
`false`	ok, `s3 disabled by declaration`	**no** `s3` block, so the Service skips export by design

S3_ENABLED	precheck	the file gets
`true`, a value missing or misshapen	**FAIL**, naming each such variable	nothing
`true`, key rejected	**FAIL** with the S3 error code (`InvalidAccessKeyId`, `SignatureDoesNotMatch`, `NoSuchBucket`, `PermanentRedirect` for a wrong region)	nothing
`true`, key valid	ok, with bucket, region and the key id's `sha8`	the full block

The proof is a SigV4-signed `ListObjectsV2` with `max-keys=0`, so it writes nothing. `AccessDenied` counts as success: AWS checks the signature before the policy, and a writer key with `PutObject` alone is exactly what this bucket should be given.

The Service gates export on the bucket alone, so a bucket with empty keys would score every run, attempt every upload, and log `S3 export failed` where no installer looks (§7). That is why the declaration is required and proved rather than defaulted. `--copy-from` carries only `workload_llm`.

To move an existing install's keys into an env file without printing them, run this. It reads from the live Secret, since on some clusters that is the only copy:

```
kubectl -n rossoctl-system get secret autobench-instances -o json \
  | jq -r '.data["<encoded-iss-host>.json"]' | base64 -d \
  | ( umask 077; jq -r '.s3 // error("no s3 block") | "S3_ENABLED=true", "S3_BUCKET=\(.bucket)",
       "S3_REGION=\(.region)", "S3_ACCESS_KEY_ID=\(.access_key_id)",
       "S3_SECRET_ACCESS_KEY=\(.secret_access_key)", (.prefix // empty | "S3_PREFIX=\(.)"),
       (.endpoint_url // empty | "S3_ENDPOINT_URL=\(.)")' >> ~/.rossoctl-
   <cluster>/autobench.env )
```

Then, on either platform:

```
kubectl -n rossoctl-system create secret generic autobench-instances \
  --from-file="<encoded-iss-host>.json=instances/<encoded-iss-host>.json" \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl -n rossoctl-system rollout restart deploy/autobench-service
```

The filename encodes the `iss` **host** with `:` rewritten to `_`; the exact `iss` inside the file is the source of truth. The Service reads instance files at startup, hence the restart. Note that `PUT /config` is in-memory only — a correction that must survive a restart goes in the Secret. It merges field by field, so a body may carry only what you are changing.

Four properties of these scripts are load-bearing:

- **Values reach `jq` through the environment, not `--arg`.** argv is world-readable (`ps`, `/proc/<pid>/cmdline`); a process's environment is not.
- **The file is created `600` by `umask`,** not `chmod`ed afterwards, so there is no window in which credentials sit world-readable on disk.
- **Facts a script cannot discover are carried or declared, never guessed.** `--copy-from` carries `workload_llm` over from an existing file, so a cluster rebuild reproduces it instead of losing it. `s3` is declared through `S3_*` (above) rather than copied. Losing either one is silent, and losing `s3` costs every run its artifacts.

- **A no-auth MLflow still needs a `bearer_token`,** which is not a contradiction: the Service mints a token *before* it reads, and with neither a bearer nor client-credentials the token helper raises, the route catches it, and the run fails soft into an empty token report. Both bootstrap scripts therefore emit the same ignored placeholder, `unused-no-auth-reader`, when the MLflow is declared unauthenticated (§3.7) — it is what KinD's reader gets. Writing *no* credential key is the one thing that does not work, and `GET /mlflow/health` reports it as a failed `auth` stage rather than leaving you to infer it from an empty report.

5. Installing with Helm

```
deploy/helm/autobench/
  Chart.yaml            version = chart version; appVersion = the default image tag
  values.yaml           the two platform shapes, documented inline
  templates/            deployment, service, route, httproute, ibac-judge, ibac-judge-hooks,
                        _helpers.tpl, NOTES.txt
  files/                proxy.py (the judge), platform-config-ibac.py (its two lifecycle hooks)
deploy/helm/values-openshift.yaml   the OpenShift shape's overrides
deploy/helm/values-kind.yaml        KinD's overrides
```

`platform` selects the shape and is the only value most installs need to think about. It drives the pod security context and which ingress object is rendered (§3.4). The chart **references but never creates** `autobench-instances`.

Parity with the raw manifests is the chart's correctness gate, and it has teeth. Run this after any template change:

```
helm lint deploy/helm/autobench
python3 reference/helm-parity-check.py     # 12 checks; "Chart and manifests agree."
```

It renders both platform shapes and diffs them against `deploy/{deployment,service,kind/httproute}.yaml` and against the OpenShift patched render. The standard `app.kubernetes.io/*` labels are deliberately **absent**: the sole label and selector is `app: autobench-service`, because that is what the live Deployments select on, and a Deployment's selector is immutable. If the check fails, the chart is wrong — not the manifests.

`reference/autobench-install.sh` is the install, on either platform. It runs every step below in order and stops at the first one that is wrong:

1. tools;
2. inputs;
3. `preflight.py --pre-install` (it must report 0 failures). What the release itself creates is not required to exist yet: the trace-writer RoleBinding, and with the judge its Deployment, its upstream Secret and the `ibac.*` fields;
    - 3b. **the MLflow read path** — on KinD `mlflow-reader` and the collector repoint (§5.2); on OpenShift the `mlflow-reader` ServiceAccount and its token Secret, whichever is missing (§5.1);
    - 3c. on KinD only, **CoreDNS's cache stops caching a SERVFAIL** (§5.2);
4. the bootstrap script, which writes the instance file;
5. the `autobench-instances` Secret, with only that one key replaced and the others kept and named;

6. `helm upgrade --install --wait`;
7. a `rollout restart`, if the Deployment already existed;
8. verification: `/healthz`, the `s3` block read back from the live Secret, and `preflight.py` again, this time with the MLflow round trip.

```
reference/autobench-install.sh --env-file ~/.rossoctl-ykt3/autobench.env --dry-run   # checks
   only
reference/autobench-install.sh --env-file ~/.rossoctl-ykt3/autobench.env
```

`--dry-run` runs steps 1–4 for real and prints 3b and 5–7 instead of running them. They are the only steps that write to the cluster. Inputs are §3.1's. A dry-run against a cluster that still has the release proves nothing about a *fresh* install, because the chart's own objects are still there: the first real install after an uninstall is the case that matters.

If the live release runs the IBAC judge, an install that does not ask for the judge refuses to run. Pass `--ibac-judge` to keep it, or `--no-ibac-judge` to remove it. Losing the judge on an upgrade is otherwise silent until the plugin legs fail.

The numbered subsections below are the manual equivalent: what the script runs, for when you need a single step.

What the scripts install: one cluster

`autobench-install.sh` and `autobench-uninstall.sh` automate **one layout: the Service and its workloads on the same cluster.** One run of the installer puts everything in §1's table onto the cluster its `KUBE_CONTEXT` names:

- the `autobench` release in `rossoctl-system`;
- an instance file for that cluster's own `iss`, with the endpoint templates left `null` because the Service reaches the workloads over in-cluster DNS;
- the MLflow read path the instance file needs;
- with `--ibac-judge`, the judge;
- on KinD, the CoreDNS fix.

Running it on a second cluster gives that cluster its own, independent install.

A cross-cluster service–workload setup is supported by the Service, but beyond the scope of these scripts. In that layout the Service runs on one cluster and benchmarks workloads on another. Nothing in the Service changes; what the scripts would have to do differently is provisioning, and today it is done by hand. It needs:

- **An instance file keyed to the workload cluster's issuer**, added to the Service's `autobench-instances` Secret next to the Service's own. It carries:
  - that cluster's `rossoctl_base_url`;
  - `mcp_endpoint_template` and `agent_endpoint_template` set to the workload cluster's Route shape (`https://{service}-{namespace}.apps.<cluster>.example.com`);
  - `workload_otel` aimed at the workload cluster's collector;
  - an `mlflow` block naming the MLflow beside the Service.

- **A collector on the workload cluster** that forwards to that MLflow, with the bearer token and `x-mlflow-*` headers the MLflow requires.
- **The IBAC judge on the workload cluster**, because AuthBridge consults its own cluster's `rossoctl-platform-config`.
- **Callers that take their token from the workload cluster's Keycloak**, since the issuer is what selects the instance.

The rationale and the residual infrastructure requirements are in `SERVICE_DESIGN_DECISIONS.md` ("per-instance workload endpoints" and "Getting the agent's own LLM/tool spans").

5.1 OpenShift

```
python3 reference/preflight.py --platform openshift --context <ctx> \
        --password-file ~/.rossoctl-ykt3/benchmarker.pass                  # 0 failures first
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context <ctx> -f deploy/helm/values-openshift.yaml
oc -n rossoctl-system rollout status deploy/autobench-service
HOST=$(oc -n rossoctl-system get route autobench -o jsonpath='{.spec.host}')
curl -fsS "https://$HOST/healthz"                                          # {"status":"ok"}
```

Leave `route.host` empty unless you need a specific name — the router generates `autobench-<namespace>.apps.<cluster>`, which is the shape every recipe in the developer guide assumes.

Before the bootstrap, the default MLflow credential has to exist: the `mlflow-reader` ServiceAccount, which the chart's trace-writer RoleBinding names but never creates, and the token Secret `mlflow-reader-token` the bootstrap copies into the instance file. `autobench-install.sh` runs `reference/ocp-mlflow.sh install --context <ctx>` as step 3b, which creates whichever of the two is missing and writes `cm/autobench-mlflow-credential` **before** it does, naming each object it made and its uid. Uninstall reads that record and deletes only what it names, and only while the uid still matches. An account that was there first is reused and left in place, and `--install-mlflow always` takes it over instead. A RoleBinding is never deleted: one made by hand that still names the account is reported. Nothing is created at all when the install declares another credential shape (the table in §3.7), or with `--install-mlflow never`.

5.2 KinD

DEV/TEST only. On a **freshly rebuilt** cluster do not start here: `reference/kind-post-setup.sh` builds and `kind load`s the image, re-seeds the `benchmarker` user, ensures the team secrets, generates the instance config and creates the Secret. The chart is the last step of that, and alone it installs a pod with nothing to authenticate as.

```
IMAGE=ghcr.io/rossoctl/autobench:v1.36 reference/kind-post-setup.sh    # first time / after a
   rebuild
python3 reference/preflight.py --platform kind --context kind-rossoctl \
        --password-file ~/.rossoctl-kind/benchmarker.pass
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context kind-rossoctl -f deploy/helm/values-kind.yaml
curl -fsS http://autobench.localtest.me:8080/healthz                  # {"status":"ok"}
```

The installer owns KinD's MLflow read path. The MLflow `rossoctl-deps` ships is OIDC-gated and refuses both the collector's export and the Service's read (§8), so on KinD `autobench-install.sh` (step 3b, via

`reference/kind-mlflow.sh`) applies `deploy/kind/mlflow-reader.yaml`, waits for its traces API to answer, and repoints the collector at it. `INSTALL_MLFLOW` / `--install-mlflow` choose:

mode	does
`auto` (default)	installs unless a Service already serves `MLFLOW_URL`. One that does — say, a reader `kind-post-setup.sh` made — is **reused and never owned**: uninstall leaves it
`always`	installs even then, and **takes ownership**
`never`	touches nothing

Ownership is a record, not a guess: before changing anything the install writes `cm/autobench-mlflow-install` in `rossoctl-system`, holding the collector's config as it was and its sha8 (the config's credentials are `${env:…}` references, so the copy carries no secret). `autobench-uninstall.sh` acts only when that record exists:

- the collector config is put back **only if it is still exactly what the install left**; if someone changed it since, it is left alone and the recorded original is saved next to the uninstall's manifest record instead;
- then `mlflow-reader` is deleted and verified NotFound, and the record goes. The traces themselves live in the platform's postgres and survive.

`--keep-mlflow` on the uninstall skips all of it.

The installer also stops KinD's CoreDNS caching a failed lookup (step 3c, via `reference/kind-dns.sh`). A KinD node forwards DNS to its container runtime. On podman for macOS that resolver now and then answers SERVFAIL, and CoreDNS caches the answer for 5 s by default. So the resolver's retry, and the agent's second probe 0.3 s later, both get the cached failure, and the task fails in under a second as `model_probe_failed`. That is how the 1-task smoke test was lost on 2026-10-04. The fix is one line in the `.:53` server's `cache` block, `servfail 0`, so a retry goes upstream again. `reload` applies it in place and nothing restarts.

Measured in a controlled A/B, with the same CoreDNS image and an upstream that fails 5% of queries:

Corefile	lookups lost (failed, and the retry too)	upstream queries
kind's own	33 of 1500	1,100
with `servfail 0`	**0** of 1500	6,100
with `servfail 0` + `serve_stale 1h`	0 of 1500	**893,400**

The extra upstream load comes from the search suffixes the node inherits, which always fail and are now asked again each time. `serve_stale` is deliberately **not** used: the podman resolver answers with TTL 0, and against that serve_stale refreshes on every request.

`KIND_DNS` / `--kind-dns` choose `auto` (the default: harden unless the cache block already has `servfail 0`) or `never`. Ownership works like the MLflow record: the install writes `cm/autobench-dns-install` in `kube-system`, holding the Corefile as it was. A Corefile already hardened some other way gets no record and is never touched. `autobench-uninstall.sh` restores the original **only if the Corefile is still exactly what the install left**. Otherwise it saves the original next to its manifest record and changes nothing. `--keep-dns` skips the step.

`8080` there is the **host** port KinD publishes the istio gateway on; it is not this Service's port, and the two matching is a coincidence. Note also that `kind load docker-image` bypasses the registry, so a KinD pod's digest will not match the published one even at the same tag — compare source, not digests, there.

5.3 Adopting an install made from the raw manifests

A chart cannot replace an existing Deployment — the selector is immutable — but it can take ownership of one. Both clusters that predate the chart were adopted this way rather than reinstalled, which is why neither had an outage: **the running pod is not recreated.**

```
for obj in deploy/autobench-service svc/autobench-service httproute/autobench; do   # or
    route/autobench
  kubectl -n rossoctl-system annotate "$obj" \
    meta.helm.sh/release-name=autobench meta.helm.sh/release-namespace=rossoctl-system --
    overwrite
  kubectl -n rossoctl-system label "$obj" app.kubernetes.io/managed-by=Helm --overwrite
done
helm upgrade --install autobench deploy/helm/autobench -n rossoctl-system -f <values>
```

Without the annotations Helm refuses the install with "invalid ownership metadata". Afterwards, confirm the pod's `restartCount` and `creationTimestamp` are unchanged — adoption should be a metadata-only operation.

Read release ownership from the Secrets, not from `helm list`. A `helm list --kube-context` comparison across two clusters has returned identical output for different clusters here; the release objects cannot lie about where they live:

```
kubectl -n rossoctl-system get secret -l owner=helm,name=autobench --context <ctx>
# sh.helm.release.v1.autobench.v1
```

5.4 The IBAC judge — installed and uninstalled with the release

Needed **only** by the plugin legs of the matrix (#5–#8). Skip it entirely if you are not running those; nothing else in AutoBench calls it.

The judge is AutoBench's, not the platform's. Rossoctl ships none — `rossoctl-platform-config` arrives with every `ibac.*` field empty — and a cluster with no benchmark workloads never needs one. So the chart installs it and `helm uninstall` removes it, which is the whole point: no leftover Deployment for the next person to find and wonder about.

With `autobench-install.sh` this is `--ibac-judge` plus three variables:

- `IBAC_JUDGE_KEY_FILE`, which becomes the Secret below;
- `IBAC_JUDGE_UPSTREAM_BASE`;
- `IBAC_JUDGE_MODEL`.

The script shape-checks the last two before it writes anything, because each has already failed every judge call while passing a presence check. The base must be a **base** URL (the plugin appends `/v1/chat/completions`), and the model takes **no** `openai/` prefix. By hand:

```
# The upstream key first. It is a credential, so the chart references it and never creates it —
    and
```

```
# for the same reason does not delete it. It must be a key for the SAME LLM service the
   workloads
# already call (§2): the judge is one more caller of it, and no two services share a key table.
umask 077; kubectl -n rossoctl-system create secret generic ibac-judge-upstream \
  --from-file=apikey=<that profile's key file>
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context <ctx> -f deploy/helm/values-openshift.yaml \
  --set ibacJudge.enabled=true \
  --set-string ibacJudge.upstreamBase="$INTERNET_LLM_API_BASE" \
  --set-string ibacJudge.model="$INTERNET_LLM_MODEL" --wait
```

`upstreamBase` and `model` are required and appear in no committed values file, because one is a live host and this repo is public. Pass them from the selected profile (§2). `model` must be an id **that gateway lists in `GET /v1/models`** — a model the judge's key cannot see fails one judge call at a time, mid-leg, and the plugin falls back to admitting the call.

What is chart-owned and what is not. The judge Deployment, Service and code ConfigMap are ordinary release objects. The two `ibac.*` fields are not objects at all: they are keys inside `rossoctl-platform-config`, which is templated by the **`rossoctl` release**. Helm owns whole objects, never fields inside another release's, so the chart handles those two keys with a pair of hooks — `post-install`/`post-upgrade` records what they said in `configmap/ibac-judge-prior` and patches them; `pre-delete` writes the recorded pair back. Do not delete that record by hand: without it the uninstall leaves the fields alone rather than guess, and they then name a judge that no longer exists.

object	created by	removed by `helm uninstall`
`deploy/ibac-judge`, `svc/ibac-judge`, `cm/ibac-judge-code`	the chart	yes
`sa/role/rolebinding ibac-judge-config`	the chart	yes
`secret/ibac-judge-upstream`	you, out-of-band	**no** — delete it yourself when done
`ibac.judgeEndpoint` / `judgeModel` in `rossoctl-platform-config`	post-install hook	restored to their previous values by the pre-delete hook

Two failure modes to know, both silent:

- **A `helm upgrade` of the `rossoctl` release reverts the patch.** That ConfigMap is re-rendered from the platform chart's own values, so the judge fields go back to empty. The plugin then loads and does nothing: every tool call admitted, legs #5–#8 still pass, and the only tell is a judge call count of zero. Re-run `preflight.py --plugin-legs` after any platform upgrade — that is why the check reads the live cluster rather than trusting install-time state.
- **A failed `pre-delete` hook aborts the uninstall.** Deliberate: the alternative is deleting the judge while the platform still points at it. Read the Job's log, fix it, retry — or `helm uninstall --no-hooks` and put the fields back by hand.

Verify both directions:

```
kubectl -n rossoctl-system get cm rossoctl-platform-config \
  -o jsonpath='{.data.config\.yaml}' | grep -A6 '^ibac:'      # judgeEndpoint/judgeModel now set
kubectl -n rossoctl-system get cm ibac-judge-prior -o jsonpath='{.data}'   # the recorded prior
   pair
```

```
kubectl -n rossoctl-system logs job/ibac-judge-config-apply   # and how it got there
```

The apply Job is **not** deleted on success — its log is the only account of what the fields said at the moment of the patch, and the next upgrade replaces it. The restore Job is the opposite: it deletes itself on success, because after an uninstall nothing else ever would.

Enforcement itself is proven by **counting judge calls**, never by a pass rate — an inert plugin and a working one produce the same pass rate. See `docs/PLUGIN_OVERHEAD.md`.

5.5 Upgrade, rollback, uninstall

```
helm upgrade autobench deploy/helm/autobench -n rossoctl-system -f <values> \
  --set image.tag=v1.36
helm history  autobench -n rossoctl-system
helm rollback autobench 1 -n rossoctl-system
helm uninstall autobench -n rossoctl-system          # leaves autobench-instances behind
```

`helm uninstall` deletes only what the chart owns, so the instance Secret survives — which is what you want, since regenerating it means re-reading credentials. The image tag is immutable, so `imagePullPolicy: IfNotPresent` is correct and an upgrade means changing `image.tag`, never restarting to pick up a rebuild.

With `ibacJudge.enabled` the uninstall does more: it runs the `pre-delete` hook that puts `rossoctl-platform-config`'s `ibac.judgeEndpoint` / `judgeModel` back to what they were before the install, then deletes the judge along with everything else (§5.4). Two consequences. A failed hook **aborts** the uninstall rather than leaving the platform pointed at a judge that is about to go — read `logs job/ibac-judge-config-restore`, or use `--no-hooks` and fix the fields by hand. And `secret/ibac-judge-upstream` survives, like the instance Secret and for the same reason: it is a credential the chart never created.

`helm rollback` does **not** re-run the pre-delete hook, and a rollback across an `ibacJudge.enabled` boundary is the one case worth avoiding — roll forward with an explicit `helm upgrade` instead, so the apply hook reconciles the fields.

`reference/autobench-uninstall.sh` is the uninstall. It takes the same env file, and it does the parts that a bare `helm uninstall` leaves to you. Its scope is the installer's: one cluster (§5, "What the scripts install"). Step 1 lists workloads on the Service's own cluster, so in a cross-cluster setup, tear the remote workloads down through the Service *before* uninstalling it.

```
reference/autobench-uninstall.sh --env-file ~/.rossoctl-ykt3/autobench.env --dry-run   # lists
   only
reference/autobench-uninstall.sh --env-file ~/.rossoctl-ykt3/autobench.env [--teams team1,team2]
```

1. **Workloads first, through the Service.**
   - Every `exgentic-*` AgentRuntime in the team namespaces is deleted by exact name: agents before tools, with `autobench-cli delete-agent` / `delete-tool`. These log in as `benchmarker`, with `KC_SERVICE_PASSWORD[_FILE]`.
   - The script then re-lists until none remain, and does **not** uninstall if any survive.
   - Anything not named `exgentic-*` is never touched.
   - Why not `autobench-cli teardown`: `DELETE /benchmarks/{b}/deploy` defaults `experiment=default`, and for a named experiment it returns 204 having deleted nothing. It also stops at a missing agent before it reaches that agent's MCP tool.

2. **Record** `helm get manifest` and `helm get hooks` under `/tmp/autobench-uninstall-<ctx>-<ts>/`. This is the authority on what the release owns, and the only list left once the release is gone.
3. **`helm uninstall --wait`.** If the judge's pre-delete hook fails, the script stops and names the Job's log. It never reaches for `--no-hooks`.
4. **Verify.**
   - Every object in the saved manifest must be NotFound. One of them, the trace-writer RoleBinding, lives in the MLflow namespace and not in `rossoctl-system`.
   - `rossoctl-platform-config` must no longer name the judge.
5. **Report the two out-of-band Secrets**, `autobench-instances` and `ibac-judge-upstream`, by key name. They are kept unless you pass `--purge-secrets`.
6. **The MLflow read path**, if and only if `autobench-install.sh` made it. On KinD the collector is restored and `mlflow-reader` removed (§5.2). On OpenShift the recorded `sa/mlflow-reader` and its token Secret are deleted, and any RoleBinding still naming the account is reported, never deleted (§5.1). `--keep-mlflow` skips this step.
7. **CoreDNS's Corefile** on KinD, put back byte for byte, if and only if `autobench-install.sh` changed it and nothing has changed it since (§5.2). `--keep-dns` skips this step.

6. Verifying the install

`/healthz` proves the pod runs. It proves nothing about the chain the pod exists to drive, and every link in that chain has a failure mode that still returns 200. Work outward:

```
TOKEN=…   # ROPC; see DEVELOPER_GUIDE.md §3
curl -fsS "$BASE/healthz"
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks" | jq '.items[].name'
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks/gsm8k" | jq
   '.agents[0].container_image'
```

`GET /benchmarks` returns **`items`**, not `benchmarks` — that has broken scripts twice — and it only summarises each benchmark (`name`, `mcp_image`, `agents`, `default_model`). An agent's `container_image`, `extra_env` or `model_override` is visible only through `GET /benchmarks/<name>`, so confirming a catalog change against the list endpoint alone can pass while the thing you changed is still the old value.

Then the only check that means anything: **a 1-task gsm8k leg whose report carries a non-zero token row.**

```
uv run autobench-cli --base "$BASE" all --benchmark gsm8k --tasks 1 --teardown   #
   DEVELOPER_GUIDE.md §7.1
```

what you look at	expected	what it proves
run status	`completed`, pass rate 1.0	Service → Rossoctl → operator → agent → MCP → evaluator
tokens on the task row	non-zero (~320 in / 87 out)	agent → collector → MLflow → Service, the whole telemetry chain
`model` on the row	the gateway's model id, not `"unknown"`	the Service read the experiment the collector wrote to

what you look at	expected	what it proves
pod `imageID` digest	matches the tag you installed (OpenShift)	the pod is not serving a stale `:latest`

A zero-token row is the single most informative failure in this system, because it is the shared signature of four unrelated causes: a refused MLflow read, an experiment-id mismatch, a collector that cannot be reached, and `workload_agent_runner: service` on a leg run at `max_parallel_sessions`

> 1 (§3.4). Preflight separates the first three and warns on the fourth. Only a run confirms the fourth, and only in part: it zeroes some rows and moves other rows' tokens onto the wrong task.

7. Symptoms that lie

symptom	actual cause	how to confirm
run passes, every token count 0, `model: "unknown"`	refused MLflow read, wrong experiment id, unreachable collector	§2.4, then §6
a p>1 leg passes, but **some** rows show `llm_count 0` and others carry a token count that belongs to a different task	`workload_agent_runner: service`. Its threads read a process-wide trace context owned by the last-started session, so spans are parented to the wrong task or orphaned. p=1 legs are unaffected, which is what makes it look intermittent	the instance's `workload_agent_runner` (`preflight.py` warns on it). On gsm8k compare per-task input tokens with an earlier run: they are per-task constants. §3.4
run passes `pass_rate 1.0`, and `report.ndjson`/`token_report.ndjson` are **zero bytes** — only 4 of the 8 artifacts carry anything	the Service's *own* span export failed, so MLflow never got the root `Agent.Session` span and the trace was dropped. On OpenShift, one of the two halves in §3.4: no service-CA trust anchor (TLS) or no trace-writer RoleBinding (403)	`autobench-cli mlflow-health` (§3.7) — its `write` stage names the cause where the run named nothing, and it is the only check that reproduces this without a run. `preflight.py` reports both halves separately as well
reports empty, or every token count 0, **right after someone set `insecure_tls: false` to harden the install**	the MLflow **read** client has no trust anchor — only the OTLP write path does. `false` leaves it on the system trust store, which lacks the service CA	`autobench-cli mlflow-health`: `write` stays ok while `read`/`round_trip` FAIL with `CERTIFICATE_VERIFY_FAILED`. That split is the signature. §3.4
runs score normally, `artifacts` is empty, and the Service logs `S3 export failed`	the instance file names a bucket with **empty or wrong keys**: the Service gates export on the bucket alone, so it attempts every upload. A hand-written instance file, or one made by a script older than the S3 declaration, can carry exactly that	read the block back as lengths, never values: §4's `get secret … | base64 -d` piped into `jq '.s3 | map_values(length)'` — a `0` beside a key is the cause. Regenerate with `S3_ENABLED` declared; the bootstrap now proves the key before writing, and `S3_ENABLED=false` writes no block at all

symptom	actual cause	how to confirm
the run publishes **no artifacts at all**, `botocore … SSLError: unable to get local issuer certificate`	`REQUESTS_CA_BUNDLE` was used for the service CA instead of `OTEL_EXPORTER_OTLP_TRACES_CERTIFICATE`; botocore honours it too and the S3 client can no longer validate AWS's cert	§3.4
a task's `error` in the **published** `run.json` reads `other (shape 7b2e…)` and says nothing about what failed	the S3 artifacts classify error text to a closed category set rather than publishing it (the bucket is anonymously listable); `other` means the message matched no known category	the verbatim text was never discarded — `GET /benchmarks/<b>/runs/<id>` on the authenticated API still returns it, and the Service logs it at **WARNING** beside the same shape id. Section 6.6 of the developer guide; add a category to `public_errors._CAUSES` once you know the cause
one task errors `A2A task ended in state 'failed': Error: timed out` at ~30 s, with `llm_count: 0`	the LLM gateway accepted the connection and never answered, **and** the instance runs `workload_agent_runner: service`. That runner caps a single `react` at a hard-coded **30 s** with no retry (`docker` and `venv` allow 600 s), so a stalled completion becomes a failed task. Under the default `direct` there is no RPC hop: litellm's own timeout fires and the agent retries, so a stall costs time rather than the task. On appworld this was 12–16 of 20 tasks a leg under `service`	the instance's runner first; then probe the gateway from inside the agent pod: a stall is a *read* timeout after TLS succeeds, and it also hits the unauthenticated `GET /public/litellm_model_cost_map`, which proves it is not the model
`/deploy` returns **502**	Keycloak or the operator returned 403 — the service credential no longer logs in, or the user lacks the `rossoctl-operator` realm role. The install itself looks healthy: the password is only used per deploy	`preflight.py --password-file …` (§3.6); then the role mapping in the realm
`/deploy` returns **424**, agent `CrashLoopBackOff`	collector endpoint on 4318, or `hf-secret` missing so the MCP pod never started	the collector's `command`; `get secret hf-secret`
token request 400 `Account is not fully set up`	the realm requires `firstName`/`lastName` for ROPC	the user's profile
tasks error on the first completion, run finishes with zeroes	`openai-secret` empty, or holding the *other* gateway's key	`preflight.py` prints both namespaces' `sha8`; compare them
on KinD, a task fails in under a second as `model_probe_failed`, the API's text saying `Temporary failure in name resolution`	the node's resolver answered SERVFAIL once, and CoreDNS cached it for 5 s, so the agent's retry got the same answer	`kubectl -n kube-system get cm coredns -o jsonpath='{.data.Corefile}'`: the `.:53` block's cache needs `servfail 0`. `reference/kind-dns.sh install` adds it (§5.2)

symptom	actual cause	how to confirm
`Model endpoint … is unreachable` on every task	the agent pod is running **dev145**, whatever `:latest` points at	the wording: dev146 says `did not respond … after N attempt(s)`. **Read it from the authenticated API, not the S3 artifacts** — those now classify both wordings to the same `model_probe_failed`, so the distinction is gone there. Compare the pod's `imageID` digest, never the tag
a config change appears to do nothing	`InstanceConfig` and the `SERVICE_*` settings both use `extra="ignore"` — an unknown field is dropped **silently**	read the value back; adding one is never self-verifying
the new `api_base` is ignored	a stale Deployment from an earlier bring-up is still serving the old one	list Deployments in the team namespaces by age
a pod serves old code at the right tag	`:latest` agent/MCP images: a *running* pod never re-pulls	tear down before deploying; compare `.status.containerStatuses[].imageID`
`mlflow-reader` restarts ~30 s after boot with no error in the log	OOMKilled — one MLflow 3.x worker idles at ~2.3 GiB, so a 2Gi limit is below the floor	the container's `lastState`, not its log
`get crd routes.route.openshift.io` says Routes are missing	Routes come from the aggregated apiserver and have no CRD	`kubectl api-resources --api-group=route.openshift.io`
`/healthz` 503, deploy 500 `httpx.ReadError`, agent CrashLoop — all at once, on KinD	a ztunnel restart un-enrolled older pods from the ambient mesh	recycle every pod older than ztunnel's Ready condition, in **all** namespaces

8. Appendix: the raw-manifest path, and two KinD-only objects

The manifests under `deploy/` remain supported and are what the chart is checked against:

```
kubectl apply -f deploy/service.yaml -f deploy/deployment.yaml
oc patch deploy autobench-service --patch-file deploy/openshift/deployment-patch.yaml   # OpenShift
kubectl apply -f deploy/kind/httproute.yaml                                             # KinD
```

On OpenShift the patch must go on **before** the rollout, not after: it is what removes the pinned UID that restricted-v2 would otherwise reject.

Two objects live under `deploy/kind/` and are in neither the chart nor the OpenShift path, because neither has an OpenShift counterpart.

`mlflow-reader.yaml` — stock MLflow, no auth, serving the *same* postgres as the writer. It exists because the MLflow `rossoctl-deps` installs runs mlflow-oidc-auth, which rejects both the Service's read and the

collector's write; the run then publishes zeros on every task, which reads like missing telemetry rather than a refused read. It is a "reader" by convention only. Two of its settings are measured rather than chosen: `--workers 1` (MLflow 3.x defaults to four) and a **4Gi** limit. Gate on a 200 from `/api/2.0/mlflow/traces` rather than on the pod going Ready — two pip installs run at container start — and pass `experiment_ids`, since that endpoint answers 400 without it. Probe from **inside** the pod (MLflow 3.x rejects the API server's service proxy as a DNS-rebinding attempt) with `python`, not `curl`, which the image does not ship. `autobench-install.sh` does all of this on KinD, including pointing the collector at the reader, and records that it did so the uninstall can undo it (§5.2); `kind-post-setup.sh` does it too, but without the record. By hand it is:

```
python3 reference/kind-collector-mlflow.py           # patch + restart; idempotent
python3 reference/kind-collector-mlflow.py --check   # report only, exit 1 if unpatched
```

`ibac-judge.yaml` — the upstream proxy the AuthBridge `ibac` plugin calls. The plugin's ConfigMap is cleartext, so the key cannot live there; this holds it in a Secret and adds the `Authorization` header outbound, leaving the plugin pointed at a credential-free in-cluster URL. Without it the plugin is **inert**: it admits every call and the run looks like a clean pass, which is why enforcement is proven by *counting judge calls* and never by reading a pass rate. `UPSTREAM_BASE` in the committed copy is an `example.com` placeholder — substitute the real gateway at apply time, and `rollout restart` after the key lands, since the proxy reads it once at pod start.